

UNIVERSIDAD SERGIO ARBOLEDA

**LABORATORIO DE BUSQUEDA INFORMADA CON PAC-MAN:
ALGORITMO A* Y DISEÑO DE HEURISTICAS**

Inteligencia Artificial (SIST5036)

Grupo 4

Juan David Andradé Gómez

Mario Jiménez López

Camilo Andrés Díaz García

*Universidad Sergio Arboleda
Escuela de Ciencias Exactas e Ingeniería
Bogotá D.C., 2026*

Índice

1. Actividad 1. Exploracion del entorno	3
1.1. Procedimiento	3
1.2. De donde sale el estado inicial (9,1)	3
1.3. Componentes del problema de busqueda	4
1.4. Analisis	5
2. Actividad 2. Búsqueda de costo uniforme	5
2.1. Implementación	5
2.2. Procedimiento y resultados	6
2.3. Corrección de layout de referencia (detectada al verificar A* en la Actividad 3)	7
2.4. Hallazgo relevante: <code>python pacman.py -l mediumMaze -p SearchAgent -a fn=ucs</code>	7
2.5. Análisis	7
3. Actividad 3. Implementación de A*	8
3.1. Implementación	8
3.2. Verificación de correctitud	9
3.3. Corrección: desempate en la cola de prioridad (bug de <i>tie-breaking</i>)	9
3.4. Análisis	10
4. Actividad 4. A* sin información	10
4.1. Procedimiento y resultados	10
4.2. Pregunta de análisis	11
5. Actividad 5. A* con distancia Manhattan	11
5.1. Heurística	11
5.2. Procedimiento y resultados	11
5.3. Pregunta de análisis	12
6. Actividad 6. Distancia Euclidiana	12
6.1. Heurística	12
6.2. Procedimiento y resultados	13
6.3. Pregunta de análisis	13
7. Actividad 7. Problema de las cuatro esquinas	14
7.1. Por qué la posición sola no alcanza como estado	14
7.2. Estado elegido	14
7.3. Procedimiento y resultados	15
7.4. Hallazgo: <code>mediumCorners</code> no es utilizable en esta copia del proyecto	15
7.5. Pregunta de análisis	16
8. Actividad 8. Heurística para las esquinas	16
8.1. Punto de partida: $h(n)=0$	16
8.2. Heurística basica	16
8.3. Heurística propuesta	17

8.4. Admisibilidad	18
8.5. Consistencia	19
8.6. Pregunta de análisis (consistencia)	19
8.7. Procedimiento y resultados	19
8.8. Pregunta de análisis	20
9. Actividad 9. Experimento comparativo (esquinas)	20
9.1. Procedimiento	20
9.2. Resultados	20
9.3. Factor de reducción de expansiones	21
9.4. Análisis	21
10. Actividad 10. Búsqueda de todos los alimentos	21
10.1. El problema	21
10.2. Por qué crece tanto el espacio de estados	22
10.3. Procedimiento y resultados	22
11. Actividad 11. Diseño de heurísticas para la búsqueda de alimentos	23
11.1. Heurística 1: distancia al alimento más lejano	23
11.2. Heurística 2: diámetro Manhattan de posición + comida restante	24
11.3. Optimización con <code>problem.heuristicInfo</code> : qué se cachea y por qué	25
11.4. Procedimiento y resultados	25
11.5. El reto del caché: qué encontramos (y por qué es un resultado honesto, no un error)	26
12. Reto: Competencia de heurísticas	26
13. Resultados generales	27
14. Análisis final	28
15. Conclusiones	30

1 Actividad 1. Exploracion del entorno

1.1 Procedimiento

Siguiendo la guia, se ejecuto el entorno de Pac-Man en modo interactivo desde la carpeta del proyecto:

```
python pacman.py
```

Pac-Man se controla con las flechas de direccion o con las teclas W, A, S, D. Se observaron la posicion inicial del agente, la ubicacion de las paredes, los alimentos disponibles y los movimientos permitidos desde cada celda.

Adicionalmente, para dejar evidencia reproducible (sin depender de la exploracion manual), se instancio programaticamente el mismo PositionSearchProblem que usa internamente SearchAgent (ver experimentos/actividad1_exploracion.py), y se imprimieron sus componentes sobre el layout tinyMaze.lay:

```
[s0] Estado inicial:      (1, 7)
[A]  Acciones disponibles en s0:
      accion=South -> sucesor=(1, 6)  costo_paso=1
[G]  Prueba de objetivo isGoalState(s0) = False
      Objetivo configurado en este problema: problem.goal = (1, 1)
[C]  Costo: 1 por movimiento
```

1.2 De donde sale el estado inicial (9,1)

El estado inicial (9,1) no se calculo a mano: sale directamente del archivo de layout pacman/layouts/mediumClassic.lay (el layout que pacman.py carga por defecto al ejecutarse sin argumentos), que dibuja el laberinto con caracteres de texto. Se muestra completo, con las 11 filas numeradas de 0 a 10 desde arriba, para poder contar x (columna, desde la izquierda) e y (fila, desde abajo) a simple vista:

```
fila 0: %%%%%%%%%%
fila 1: %o...%.....%...%
fila 2: %.%%.%.%%%%.%.%%.%
fila 3: %.%.....%.%
fila 4: %.%.%%.%  %.%.%%.%
fila 5: %.....%G  G%.....%
fila 6: %.%.%%.%%%%.%.%%.%
fila 7: %.%.....%.%
fila 8: %.%%.%.%%%%.%.%%.%
fila 9: %...%...P...%...o%
fila 10: %%%%%%%%%%
```

La letra P marca la posicion de Pac-Man. La conversion de esa posicion en el texto a coordenadas (x,y) sigue dos reglas:

- x es la columna, contada normalmente desde la izquierda empezando en 0. La P esta en la fila 9, columna 9 $\Rightarrow x = 9$.

- y es la fila, pero contada *desde abajo hacia arriba*, empezando en 0 en la ultima fila del archivo (el origen (0,0) es la esquina inferior izquierda, no la superior). El layout tiene 11 filas (indices 0 a 10 contando desde arriba); la P esta en la fila de indice 9 desde arriba, que equivale a $10 - 9 = 1$ desde abajo $\Rightarrow y = 1$.

Por eso el estado inicial es (9, 1).

Verificacion adicional con varios agentes. Este mismo layout permite confirmar la convencion con mas de un agente: las dos G (fantasmas) estan en la fila 5, columnas 8 y 11 $\Rightarrow y = 10 - 5 = 5 \Rightarrow$ posiciones (8,5) y (11,5). Esto se confirmo programaticamente (comando ejecutado desde dentro de pacman/, porque layout.py busca la carpeta layouts/ relativa al directorio de trabajo actual):

```
python -c "
import layout as layout_module
print(layout_module.getLayout('mediumClassic').agentPositions)
"
[(True, (9, 1)), (False, (8, 5)), (False, (11, 5))]
```

agentPositions es una lista de pares (esPacman, posicion): el primer elemento, con True, es Pac-Man en (9, 1) –confirmando el calculo manual de arriba–; los otros dos, con False, son los dos fantasmas en (8,5) y (11,5).

Esta conversion la hace internamente layout.py (codigo del profesor, sin modificar) al cargar el archivo .lay; getStartState() de PositionSearchProblem solo devuelve el resultado ya calculado. El mismo procedimiento aplica para leer el estado inicial de cualquier otro layout usado en el resto del informe –por ejemplo tinyMaze, usado en la seccion anterior, donde el mismo metodo da (1,7)–.

1.3 Componentes del problema de busqueda

Cuadro 1: Componentes del problema de busqueda en el entorno Pac-Man

Elemento	Descripcion
Estado (s)	Posicion (x, y) de Pac-Man dentro del laberinto.
Estado inicial (s_0)	Posicion en la que Pac-Man aparece al iniciar (depende del layout; al ejecutar <code>python pacman.py</code> sin argumentos, carga <code>mediumClassic</code> por defecto, donde $s_0 = (9, 1)$).
Acciones (A)	$\{North, South, East, West\}$; una accion solo es legal si la celda vecina en esa direccion no es pared.
Funcion sucesor (T)	<code>getSuccessors(state)</code> : por cada accion legal retorna la terna (estado sucesor, accion, costo del paso).
Objetivo (G)	<code>isGoalState(state)</code> : en <code>PositionSearchProblem</code> , llegar a la posicion configurada como meta (por defecto (1, 1)).
Costo (C)	1 por cada movimiento; el costo total de un plan es la suma de los costos de sus pasos, es decir, su longitud.

1.4 Analisis

El entorno Pac-Man encaja de forma natural en la formulacion general de un problema de busqueda $P = (S, A, T, s_0, G, C)$ presentada en la guia: el espacio de estados S corresponde a las posiciones transitables del laberinto, las acciones A estan restringidas por la geometria de las paredes, la funcion de transicion T esta implementada por `getSuccessors`, y el costo C es uniforme (1 por paso) en esta primera version del problema. Esta representacion minima –solo la posicion– es suficiente para el problema de *llegar a un punto fijo*, pero, como se vera en la Actividad 7, deja de ser suficiente en cuanto el objetivo depende de un historial (por ejemplo, “haber visitado ya tres esquinas”), porque dos estados con la misma posicion (x, y) dejarian de ser equivalentes.

2 Actividad 2. Búsqueda de costo uniforme

2.1 Implementación

Se completó `uniformCostSearch(problem)` en `pacman/search.py`. Es una búsqueda en grafo con cola de prioridad (`util.PriorityQueue`) ordenada por $g(n)$, el costo acumulado real desde el estado inicial. Se mantiene un diccionario con el mejor costo conocido por estado para evitar volver a expandir un nodo cuando ya se encontró un camino más barato hacia él (`util.PriorityQueue` no soporta *decrease-key*, así que las entradas obsoletas se descartan al extraerlas de la cola en lugar de actualizarse en el lugar).

```

80 def uniformCostSearch(problem):
81     """
82     Search the node of least total cost first.
83
84     Actividad 2. Busqueda en grafo: frontera ordenada por g(n) (costo
85     acumulado), con un diccionario bestCost para no reexpandir un estado si
86     ya se conoce un camino mas barato. El contador en cada tupla evita que
87     heapq falle al comparar estados no comparables (ver Actividad 11).
88     """
89     frontier = util.PriorityQueue()
90
91     startState = problem.getStartState()
92     contador = 0
93     # Cada elemento de la frontera es (contador, estado, acciones, g(n)).
94     frontier.push((contador, startState, [], 0), 0)
95
96     # Mejor costo g(n) conocido hasta ahora para cada estado.
97     bestCost = {startState: 0}
98
99     while not frontier.isEmpty():
100         _, state, actions, cost = frontier.pop()
101
102         # Si ya existe un camino mas barato registrado para este estado,
103         # esta es una entrada obsoleta de la cola: se descarta sin expandir.
104         if cost > bestCost.get(state, float('inf')):
105             continue
106
107         # Prueba de objetivo ANTES de expandir (evita expandir de mas).
108         if problem.isGoalState(state):
109             return actions
110
111         for successor, action, stepCost in problem.getSuccessors(state):
112             newCost = cost + stepCost
113             # Solo nos interesa este sucesor si mejora el mejor costo conocido.
114             if newCost < bestCost.get(successor, float('inf')):
115                 bestCost[successor] = newCost
116                 contador += 1
117                 frontier.push((contador, successor, actions + [action], newCost), newCost)
118
119     # Frontera vacia sin encontrar meta: no existe solucion.
120     return []

```

Figura 1: Implementación de uniformCostSearch en pacman/search.py (líneas 80-120), incluido el contador de desempate contador explicado más abajo.

2.2 Procedimiento y resultados

Se ejecutó (ver experimentos/actividad2_ucs.py) sobre mediumMaze y tinyMaze como primera línea base.

Cuadro 2: Resultado de UCS (línea base para comparar A*)

Layout	Costo	Longitud	Nodos expandidos	Tiempo (s)
mediumMaze	30	30	32	0.000270
tinyMaze (verificación)	10	10	21	0.000182
mediumClassic	12	12	69	0.000492

Estos resultados se guardaron en resultados/resultados.csv y se usarán como

línea base para comparar A* en las Actividades 4, 6 y 9.

2.3 Corrección de layout de referencia (detectada al verificar A* en la Actividad 3)

Al implementar A* (Actividad 3) y compararlo contra esta línea base, se encontró que `mediumMaze` es esencialmente un pasillo sin bifurcaciones: A* con heurística nula, Manhattan y Euclidiana expanden exactamente los mismos 32 nodos que UCS, porque no hay ningún punto de decisión donde una heurística pueda *preferir* una dirección sobre otra. Eso hace que `mediumMaze` no sirva para ilustrar el efecto de las heurísticas que piden las Actividades 5, 6 y 9.

Por eso, a partir de la Actividad 3, se adopta `mediumClassic` como layout principal para comparar UCS y A* con distintas heurísticas: tiene bifurcaciones reales y, como se muestra en la Actividad 3, allí Manhattan sí reduce los nodos expandidos frente a UCS (69 $\rightarrow$ 15). `mediumMaze` se conserva como caso de contraste ilustrativo (“¿qué pasa si el laberinto no tiene bifurcaciones?”), no como el layout principal de la comparación.

2.4 Hallazgo relevante: `python pacman.py -l mediumMaze -p SearchAgent -a fn=ucs`

Al correr ese comando de la guía tal cual, se obtienen exactamente los mismos números (costo, nodos expandidos y tiempo), pero el proceso termina con `Exception: Illegal action Stop` justo después de imprimirlos. La causa no es la búsqueda: es que los layouts de este proyecto traen muchos alimentos (no solo uno en la meta), así que al terminarse el plan de `SearchAgent` (que la guía exige no modificar) el juego sigue corriendo; `SearchAgent.getAction` devuelve entonces `Directions.STOP`, y en este motor `PacmanRules.getLegalActions` elimina explícitamente `STOP` de las acciones legales de Pac-Man. Como las métricas se calculan e imprimen dentro de `registerInitialState`, antes de que ocurra ese error, esto no afecta ningún resultado reportado en este informe; por eso los scripts de experimentos/ miden directamente sobre el `SearchProblem` en lugar de sobre el bucle completo del juego.

2.5 Análisis

UCS garantiza encontrar el camino de menor costo porque expande siempre el nodo de menor $g(n)$ conocido; en este problema, con costo uniforme de 1 por movimiento, eso equivale a expandir en orden de menor número de pasos. Al no usar ninguna heurística, UCS no tiene forma de “orientarse” hacia el objetivo: expande por igual en todas las direcciones prometedoras según el costo acumulado, sin preferir las que se acercan geométricamente a la meta. Esto explica por qué, a partir de la Actividad 4, A* con una heurística informativa expandirá menos nodos que UCS para llegar al mismo costo óptimo.

3 Actividad 3. Implementación de A*

3.1 Implementación

Se completó `aStarSearch(problem, heuristic=nullHeuristic)` en `pacman/search.py`. Reutiliza exactamente el mismo esqueleto de búsqueda en grafo que `uniformCostSearch` (Actividad 2): misma frontera `util.PriorityQueue`, mismo diccionario `bestCost` para no reexpandir un estado cuando ya se conoce un camino más barato, y la misma prueba de meta al extraer el nodo de la frontera. La única diferencia es la prioridad usada para ordenar la frontera: en UCS es $g(n)$; en A* es $f(n) = g(n) + h(n)$.

```

129 def aStarSearch(problem, heuristic=nullHeuristic):
130     """
131     Search the node that has the lowest combined cost and heuristic first.
132
133     Actividad 3. Mismo esqueleto que uniformCostSearch, con una sola
134     diferencia: la prioridad de la frontera es  $f(n) = g(n) + h(n)$  en vez de
135     solo  $g(n)$ . Con heuristic=nullHeuristic se comporta igual que UCS
136     (Actividad 4); con una heurística admisible y consistente sigue siendo
137     óptimo, pero expande menos nodos.
138     """
139     frontier = util.PriorityQueue()
140
141     startState = problem.getStartState()
142     contador = 0
143     startPriority = 0 + heuristic(startState, problem) #  $f = g + h$ ,  $g = 0$ 
144     frontier.push((contador, startState, [], 0), startPriority)
145
146     # Mejor costo  $g(n)$  conocido hasta ahora para cada estado (igual que UCS;
147     # la heurística NO se guarda aquí porque no hace falta: se recalcula al
148     # generar cada sucesor).
149     bestCost = {startState: 0}
150
151     while not frontier.isEmpty():
152         _, state, actions, cost = frontier.pop()
153
154         # Entrada obsoleta de la cola (ya se encontró un  $g(n)$  mejor): se
155         # descarta sin expandir, igual que en UCS.
156         if cost > bestCost.get(state, float('inf')):
157             continue
158
159         # Prueba de objetivo ANTES de expandir.
160         if problem.isGoalState(state):
161             return actions
162
163         for successor, action, stepCost in problem.getSuccessors(state):
164             newCost = cost + stepCost
165             if newCost < bestCost.get(successor, float('inf')):
166                 bestCost[successor] = newCost
167                 contador += 1
168                 priority = newCost + heuristic(successor, problem) #  $f(n) = g(n) + h(n)$ 
169                 frontier.push((contador, successor, actions + [action], newCost), priority)
170
171     # Frontera vacía sin encontrar meta: no existe solución.
172     return []
173

```

Figura 2: Implementación de `aStarSearch` en `pacman/search.py` (líneas 129-172). Comparar con la Figura 1: la única diferencia real es la línea de `priority = newCost + heuristic(...)`.

Cuadro 3: Los 8 requisitos de la guía y dónde se cumplen

Requisito	Dónde se cumple
1. Cola de prioridad	<code>frontier = util.PriorityQueue()</code>
2. Comenzar desde <code>getStartState()</code>	<code>startState = problem.getStartState()</code>
3. Usar <code>isGoalState()</code>	<code>if problem.isGoalState(state): return actions</code>
4. Obtener sucesores con <code>getSuccessors()</code>	<code>for successor, action, stepCost in problem.getSuccessors(state)</code>
5. Acumular el costo $g(n)$	<code>newCost = cost + stepCost, guardado en bestCost</code>
6. Calcular la prioridad $g(n) + h(n)$	<code>priority = newCost + heuristic(successor, problem)</code>
7. Evitar expansiones innecesarias	<code>chequeo if cost > bestCost.get(state, ...): continue</code>
8. Retornar una lista de acciones	<code>return actions (o [] si no hay solución)</code>

3.2 Verificación de correctitud

No hay una tabla de resultados exigida en esta actividad (esa llega en la Actividad 4 en adelante); en su lugar se verificó que la implementación es correcta comparándola contra la línea base de UCS (`experimentos/actividad3_astar_verificacion.py`):

Cuadro 4: A* comparado contra UCS en varios layouts (verificación de correctitud)

Layout	Costo	UCS exp.	A*+h=0 exp.	A*+Manh. exp.	A*+Eucl. exp.
tinyMaze	10	21	21	10	10
mediumMaze	30	32	32	32	32
mediumClassic	12	69	69	15	16
openClassic	9	63	63	27	31
trickyClassic	12	60	60	15	17

En todos los layouts el costo es idéntico (las cuatro estrategias son óptimas), A*+h=0 expande exactamente lo mismo que UCS (Actividad 4 explica por qué), y ninguna heurística expande más nodos que h(n)=0. La única excepción aparente es `mediumMaze`, que no muestra ninguna mejora con las heurísticas: como se explica al final de la Actividad 2, ese layout es un pasillo sin bifurcaciones, así que no hay ningún punto de decisión donde una heurística pueda influir. Por eso, de aquí en adelante, se usa `mediumClassic` como layout principal para comparar heurísticas (Actividades 4, 5, 6 y 9).

3.3 Corrección: desempate en la cola de prioridad (bug de *tie-breaking*)

Al preparar la Actividad 11 (heurística de comida sobre `FoodSearchProblem`), correr UCS/A* sobre ese problema hizo fallar la implementación con `TypeError: '<' not supported between instances of 'Grid' and 'Grid'`. La causa: `util.PriorityQueue` usa internamente `heapq`, que guarda tuplas (prioridad, item); cuando dos entradas empatan en prioridad, `heapq` compara item para decidir el orden. En `PositionSearchProblem`

el estado es un par de enteros (x, y) , siempre comparable, así que el empate nunca causaba error; en `FoodSearchProblem` el estado incluye un objeto `Grid` (la matriz de comida restante), que no define `__lt__` y por lo tanto no es comparable.

La corrección fue agregar un contador entero único y creciente como primer elemento de cada tupla insertada en la frontera (`(contador, estado, acciones, costo)` en vez de `(estado, acciones, costo)`, tanto en `uniformCostSearch` como en `aStarSearch`): como los contadores nunca se repiten, `heapq` nunca necesita comparar el resto de la tupla para resolver un empate, sin importar si el estado es comparable o no.

Esta corrección cambia el criterio exacto de desempate cuando dos nodos comparten la misma prioridad –antes dependía, sin quererlo, del orden en que `heapq` comparaba las tuplas `(estado, acciones, costo)`–; el algoritmo sigue siendo correcto y óptimo (el costo de la solución no cambia en ningún experimento), pero el número exacto de *nodos expandidos* sí puede cambiar, porque distintos criterios de desempate expanden nodos empatados en distinto orden. Por eso todos los números de “nodos expandidos” reportados en este informe (Actividades 2 a 6) corresponden a la versión ya corregida con el contador de desempate, re-verificados después de aplicar el fix.

Este hallazgo es, en sí mismo, una buena respuesta a una pregunta trampa típica: ¿el número de nodos expandidos es una propiedad determinista del algoritmo, o depende de detalles de implementación de la cola de prioridad? La respuesta es que el costo óptimo sí es determinista (depende solo de la heurística y el problema), pero el número exacto de nodos expandidos puede depender de cómo se rompen los empates de prioridad –un detalle de implementación, no del algoritmo en abstracto.

3.4 Análisis

Que A^* con `nullHeuristic` expanda exactamente los mismos nodos que UCS en los cinco layouts no es casualidad: con $h(n) = 0$, la prioridad de A^* se reduce a $f(n) = g(n) + 0 = g(n)$, que es exactamente la prioridad que usa UCS. Ambos algoritmos terminan expandiendo la frontera en el mismo orden. Esto confirma que UCS es, formalmente, un caso particular de A^* sin información adicional –lo que se pedirá justificar explícitamente en la Actividad 4– y es la primera evidencia de que la implementación de A^* es correcta antes de introducir heurísticas no triviales.

4 Actividad 4. A^* sin información

4.1 Procedimiento y resultados

Se ejecutaron UCS y A^* con `nullHeuristic` ($h(n) = 0$) sobre `mediumClassic`, el layout de referencia adoptado desde la Actividad 3 (ver `experimentos/actividad4_astar_nulo.py`).

Cuadro 5: UCS vs. A^* con heurística nula

Algoritmo	Costo	Expandidos	Tiempo (s)
UCS	12	69	0.000257
$A^* + h(n) = 0$	12	69	0.000222

Ambos algoritmos encuentran el mismo costo óptimo (12) y expanden exactamente el mismo número de nodos (69). La pequeña diferencia de tiempo entre ambas filas

(0.000257s vs. 0.000222s) no es una diferencia algorítmica: es ruido de medición típico de intervalos tan cortos (microsegundos), donde el sistema operativo puede interrumpir brevemente el proceso; no depende de $h(n)$, ya que ambos algoritmos ejecutan exactamente las mismas operaciones cuando $h(n) = 0$.

4.2 Pregunta de análisis

¿Qué comportamiento observa? Explique por qué A^* con $h(n) = 0$ presenta un comportamiento equivalente o muy similar a búsqueda de costo uniforme.

La prioridad que usa A^* para ordenar su frontera es $f(n) = g(n) + h(n)$. Si $h(n) = 0$ para *todo* estado n (como hace `nullHeuristic`), entonces $f(n) = g(n) + 0 = g(n)$ para cualquier nodo, sin excepción. Esa es exactamente la misma prioridad que usa UCS. Como ambos algoritmos usan la misma cola de prioridad y la misma prioridad para cada nodo, extraen los nodos de la frontera en el mismo orden, expanden los mismos nodos, y encuentran el mismo camino óptimo.

Dicho de otra forma: UCS *es* un caso particular de A^* en el que la heurística usada es siempre cero. No es una coincidencia experimental ni una casualidad del layout: es una consecuencia directa de la definición de $f(n)$, y por eso el comportamiento se repite igual en los cinco layouts probados en la Actividad 3 (Cuadro 4).

5 Actividad 5. A^* con distancia Manhattan

5.1 Heurística

`manhattanHeuristic` ya viene provista en `searchAgents.py` (no fue necesario implementarla para esta actividad):

```
236 def manhattanHeuristic(position, problem, info={}):
237     "The Manhattan distance heuristic for a PositionSearchProblem"
238     xy1 = position
239     xy2 = problem.goal
240     return abs(xy1[0] - xy2[0]) + abs(xy1[1] - xy2[1])
241
```

Figura 3: `manhattanHeuristic` en `pacman/searchAgents.py` (líneas 236-240).

Calcula $h_M(n) = |x_n - x_g| + |y_n - y_g|$: la suma de las diferencias absolutas en cada eje, que es exactamente el número mínimo de movimientos ortogonales (norte/sur/este/oeste) necesarios para llegar de n a la meta *si no hubiera paredes de por medio*.

5.2 Procedimiento y resultados

Se ejecutó A^* con Manhattan sobre `mediumClassic` y se comparó contra UCS (ver `experimentos/actividad5_manhattan.py`). En vez de una interfaz gráfica (no disponible en el entorno de ejecución usado), se registró el conjunto de celdas efectivamente expandidas por cada algoritmo (`problem._visitedlist`, la misma estructura que `graphicsDisplay.py` usaría para pintarlas en pantalla).

Cuadro 6: UCS vs. A* con distancia Manhattan

Algoritmo	Costo	Expandidos	Tiempo (s)
UCS	12	69	0.000251
A* + Manhattan	12	15	0.000068

Ambos encuentran el mismo costo óptimo (12), pero A*+Manhattan expande 4.60 veces menos nodos que UCS ($69 \rightarrow 15$): de las 70 celdas que UCS llegó a expandir, 54 nunca fueron tocadas por A*+Manhattan. Esto es evidencia directa (no solo teórica) de que la heurística está orientando la búsqueda hacia el objetivo en lugar de explorar por igual en todas las direcciones.

5.3 Pregunta de análisis

¿Por qué la distancia Manhattan puede orientar correctamente a Pac-Man aunque la heurística no considere directamente las paredes del laberinto?

Manhattan es admisible precisamente porque *ignora* las paredes: al no considerarlas, nunca puede sobreestimar el costo real, porque el camino real (que sí tiene que rodear paredes) nunca puede ser más corto que la línea de movimientos ortogonales sin obstáculos que calcula h_M . Es decir, $h_M(n)$ es siempre una *cota inferior* del costo real $h^*(n)$: en el peor caso (un laberinto muy enredado) subestima mucho y aporta poca guía, pero nunca lleva a A* a expandir un nodo equivocado creyendo que está más cerca de lo que en realidad está.

Además, aunque no vea las paredes explícitamente, Manhattan sigue *ordenando* correctamente los estados por qué tan lejos están de la meta en la métrica correcta para este problema: como Pac-Man solo se mueve en las 4 direcciones cardinales (nunca en diagonal), la distancia Manhattan es exactamente la noción de “cercanía” que corresponde a este espacio de acciones. Eso es lo que le permite priorizar, entre varios sucesores con el mismo $g(n)$, el que está en la dirección correcta hacia la meta, incluso cuando localmente esa dirección choque con una pared unos pasos más adelante (en ese caso, A* simplemente termina expandiendo también el nodo que rodea la pared, porque su $g(n)$ real sigue contando el costo verdadero recorrido).

6 Actividad 6. Distancia Euclidiana

6.1 Heurística

`euclideanHeuristic` ya viene provista en `searchAgents.py` (tampoco fue necesario implementarla):

```

242 def euclideanHeuristic(position, problem, info={}):
243     "The Euclidean distance heuristic for a PositionSearchProblem"
244     xy1 = position
245     xy2 = problem.goal
246     return ( (xy1[0] - xy2[0]) ** 2 + (xy1[1] - xy2[1]) ** 2 ) ** 0.5
247

```

Figura 4: `euclideanHeuristic` en `pacman/searchAgents.py` (líneas 242-246).

Calcula $h_E(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$: la distancia en línea recta entre n y la meta, ignorando también las paredes (por lo tanto también es admisible, por el mismo argumento que Manhattan: nunca sobreestima el costo real).

6.2 Procedimiento y resultados

Se ejecutó A* con las tres heurísticas (nula, Manhattan y Euclidiana) sobre `mediumClassic` (ver `experimentos/actividad6.euclidiana.py`).

Cuadro 7: Comparación de heurísticas con A*

Heurística	Longitud	Costo	Expandidos	Tiempo (s)
$h(n) = 0$	12	12	69	0.000262
Manhattan	12	12	15	0.000066
Euclidiana	12	12	16	0.000070

Las tres heurísticas encuentran el mismo costo óptimo (12): las tres son admisibles, así que A* sigue siendo óptimo con cualquiera de ellas. La diferencia está en cuánto ayudan a reducir la exploración: Manhattan expande 54 nodos menos que $h(n) = 0$ (69 → 15) y Euclidiana 53 menos (69 → 16). Manhattan es, además, ligeramente mejor que Euclidiana (15 vs. 16 nodos expandidos) en este layout.

6.3 Pregunta de análisis

Pac-Man se desplaza únicamente horizontal o verticalmente. ¿Cuál de las dos heurísticas representa mejor este tipo de movimiento? Justifique su respuesta utilizando los resultados experimentales.

Manhattan representa mejor el movimiento de Pac-Man, y los resultados experimentales lo confirman: expandió 15 nodos frente a los 16 de Euclidiana para llegar al mismo costo óptimo. La razón es que $h_E(n) \leq h_M(n)$ para cualquier par de puntos (la distancia en línea recta nunca es mayor que la suma de catetos ortogonales; es la desigualdad triangular aplicada a un triángulo rectángulo). Como Pac-Man solo puede moverse en las 4 direcciones cardinales —nunca en diagonal—, el costo real de moverse entre dos celdas siempre coincide con su distancia Manhattan (cuando no hay paredes de por medio) y nunca con su distancia Euclidiana, que es una subestimación adicional e innecesaria en este espacio de acciones. Dicho de otro modo: Euclidiana está diseñada para un agente que puede moverse en línea recta en cualquier ángulo, algo que Pac-Man no puede hacer; al aplicarla aquí, subestima el costo real más de lo que Manhattan lo hace, y una heurística que subestima más aporta menos información para podar la búsqueda — por eso expande más nodos.

Esta misma conclusión se repite en `openClassic` (Cuadro 4, donde la diferencia es mucho más marcada: 27 nodos con Manhattan frente a 31 con Euclidiana), lo que descarta que la ventaja de Manhattan en `mediumClassic` sea casualidad de ese layout en particular.

7 Actividad 7. Problema de las cuatro esquinas

7.1 Por qué la posición sola no alcanza como estado

Como señala la guía: dos situaciones $s_1 = (5,4)$ y $s_2 = (5,4)$ pueden corresponder a la misma posición de Pac-Man pero, si en s_1 ya visitó una esquina y en s_2 ya visitó tres, no son estados equivalentes para este problema –requieren seguir caminos distintos para llegar a la meta–. Por lo tanto el estado tiene que incluir, además de la posición, qué esquinas ya se visitaron.

7.2 Estado elegido

$$s = (\text{posición}, \text{esquinas_visitadas})$$

donde `esquinas_visitadas` es una tupla de 4 booleanos, uno por cada esquina de `self.corners` (en el mismo orden: (1,1), (1,top), (right,1), (right,top)). Se eligió una tupla –no una lista– porque el estado tiene que ser *hasheable* y *comparable* para funcionar dentro de `util.PriorityQueue` y de cualquier diccionario de costos: las tuplas de booleanos cumplen ambas propiedades sin ningún esfuerzo extra (a diferencia del `foodGrid` de `FoodSearchProblem`, que sí necesitó el contador de desempate documentado en la Actividad 3 precisamente porque no es comparable).

```

273 def getStartState(self):
274     """
275     Returns the start state (in your state space, not the full Pacman state space)
276
277     Actividad 7. El estado es (posicion, esquinas_visitadas): una posicion
278     (x,y) mas una tupla de 4 booleanos (una por esquina, mismo orden que
279     self.corners). No basta con (x,y) solo, porque dos visitas al mismo
280     punto con distintas esquinas ya marcadas no son equivalentes aqui.
281     """
282     esquinasVisitadas = (False, False, False, False)
283     return (self.startingPosition, esquinasVisitadas)
284
285 def isGoalState(self, state):
286     """
287     Returns whether this search state is a goal state of the problem
288
289     Actividad 7. Meta = las 4 esquinas visitadas, sin importar la posicion.
290     """
291     _, esquinasVisitadas = state
292     return all(esquinasVisitadas)
293
294 def getSuccessors(self, state):
295     """
296     Returns successor states, the actions they require, and a cost of 1.
297
298     As noted in search.py:
299     | For a given state, this should return a list of triples,
300     | (successor, action, stepCost), where 'successor' is a
301     | successor to the current state, 'action' is the action
302     | required to get there, and 'stepCost' is the incremental
303     | cost of expanding to that successor
304
305     Actividad 7. El sucesor es (nueva posicion, esquinas_visitadas
306     actualizado): si la nueva posicion es una esquina pendiente, se marca
307     como visitada en una tupla nueva (las tuplas son inmutables).
308     """
309

```

Figura 5: `getStartState`, `isGoalState` y el inicio de `getSuccessors` de `CornersProblem` en `pacman/searchAgents.py` (líneas 273-309, parte 1 de 2).

```

309
310     successors = []
311     position, visited = state
312     for action in [Directions.NORTH, Directions.SOUTH, Directions.EAST, Directions.WEST]:
313         # Add a successor state to the successor list if the action is legal
314         # Here's a code snippet for figuring out whether a new position hits a wall:
315         #   x,y = currentPosition
316         #   dx, dy = Actions.directionToVector(action)
317         #   nextx, nexty = int(x + dx), int(y + dy)
318         #   hitsWall = self.walls[nextx][nexty]
319         x, y = position
320         dx, dy = Actions.directionToVector(action)
321         nextx, nexty = int(x + dx), int(y + dy)
322         hitsWall = self.walls[nextx][nexty]
323
324         if not hitsWall:
325             nextPosition = (nextx, nexty)
326             nextVisited = visited
327             if nextPosition in self.corners:
328                 idx = self.corners.index(nextPosition)
329                 if not visited[idx]:
330                     nextVisited = visited[:idx] + (True,) + visited[idx + 1:]
331             nextState = (nextPosition, nextVisited)
332             successors.append((nextState, action, 1))
333
334     self._expanded += 1
335     return successors
336

```

Figura 6: Resto de `getSuccessors` de `CornersProblem` en `pacman/searchAgents.py` (líneas 309-336, parte 2 de 2): construcción del sucesor (nueva posición, esquinas visitadas actualizado).

La meta se alcanza cuando `all(esquinas_visitadas)` es verdadero –las 4 esquinas ya se visitaron–, sin importar en qué posición esté Pac-Man en ese momento. Cada sucesor recalcula la tupla completa de esquinas visitadas (las tuplas son inmutables, así que no se modifica `visited` en el lugar): si la nueva posición coincide con una esquina que todavía no estaba marcada, se genera una tupla nueva con esa esquina en `True`.

7.3 Procedimiento y resultados

Se ejecutó UCS sobre `tinyCorners` (ver `experimentos/actividad7_corners_estado.py`).

Cuadro 8: UCS sobre `CornersProblem`

Layout	Costo	Longitud	Nodos expandidos	Tiempo (s)
tinyCorners	22	22	295	0.001300

Se verificó, simulando paso a paso las 22 acciones devueltas, que el estado final es `((1,5), (True, True, True, True))`: la solución realmente visita las 4 esquinas, no solo pasa la prueba `isGoalState` por casualidad.

7.4 Hallazgo: `mediumCorners` no es utilizable en esta copia del proyecto

Al intentar usar `mediumCorners` como segundo layout de referencia (para tener un caso más grande que `tinyCorners`), UCS terminó devolviendo una lista de acciones vacía después de explorar apenas 36 nodos –sin haber visitado ninguna esquina–. Se investigó

imprimiendo directamente la cuadrícula de paredes (`state.getWalls()`) y se confirmó que el punto de partida de Pac-Man en este layout queda encerrado en un cuarto de 9×4 celdas completamente sellado por paredes en los cuatro lados, sin ninguna puerta de salida.

Para descartar que fuera un error de nuestra implementación de `CornersProblem`, se probó lo mismo con `PositionSearchProblem` (código ya implementado por el profesor, sin modificar), pidiendo un camino desde el inicio hasta la esquina $(1, 1)$: UCS también devuelve una lista vacía, confirmando que ninguna búsqueda –sin importar el problema o el algoritmo– puede escapar de ese cuarto en este layout. Por eso, igual que se descartó `bigMaze` en las primeras actividades, `mediumCorners` no se usa como layout de referencia en esta entrega; `tinyCorners` es el layout principal para las Actividades 7-9.

7.5 Pregunta de análisis

¿Por qué el estado de `CornersProblem` necesita almacenar más información que únicamente la posición (x, y) ?

Porque la prueba de objetivo de este problema no depende solo de dónde está Pac-Man, sino de *qué ha hecho* para llegar ahí –específicamente, cuáles de las 4 esquinas ya visitó–. Si el estado fuera solo la posición, dos caminos que pasan por la misma celda pero con distinto progreso (unas esquinas visitadas y otras no) se confundirían en un solo estado, y el algoritmo de búsqueda podría marcar como ‘ya explorado’ un estado que en realidad todavía necesita explorarse con otra combinación de esquinas pendientes, perdiendo la garantía de encontrar la solución óptima –o incluso de encontrar alguna solución–. Codificar el progreso (esquinas visitadas) dentro del estado es lo que hace que dos situaciones con la misma posición pero distinto progreso sean, correctamente, dos estados distintos.

8 Actividad 8. Heurística para las esquinas

8.1 Punto de partida: $h(n)=0$

El archivo inicial trae `cornersHeuristic` devolviendo siempre 0. Como senala la guía, esta función es admisible por definición (nunca sobreestima nada) pero no aporta ninguna información útil para orientar la búsqueda: con ella, A^* se comporta exactamente igual que UCS.

8.2 Heurística básica

Se implemento primero la formula que sugiere la guía como primera aproximación: la distancia Manhattan a la esquina pendiente mas lejana.

$$h(n) = \max_{c \in C_p} d_M(n, c)$$

donde C_p es el conjunto de esquinas que todavía no se han visitado en el estado n (si ya no queda ninguna esquina pendiente, $h(n) = 0$).

```

350
351 def cornersHeuristicBasica(state, problem):
352     """
353     Actividad 8 -- Heuristica basica: distancia Manhattan a la esquina
354     pendiente mas lejana, h(n) = max d_M(posicion, c) para c en esquinas
355     pendientes. Admisible y consistente (ver docs/guia_codigos_clave.md).
356     """
357     position, visited = state
358     corners = problem.corners
359     pendientes = [c for c, v in zip(corners, visited) if not v]
360     if not pendientes:
361         return 0
362     return max(abs(position[0] - c[0]) + abs(position[1] - c[1]) for c in pendientes)
363
364

```

Figura 7: cornersHeuristicBasica en pacman/searchAgents.py (líneas 351-364).

8.3 Heuristica propuesta

Siguiendo la sugerencia de la guia de analizar “si es posible disenar una heuristica mas informativa”, se generalizo la heuristica basica con el mismo patron de diametro que ya se habia usado en foodHeuristic (Actividad 11): en vez de mirar solo la distancia de Pac-Man a la esquina pendiente mas lejana, se considera el diametro –la mayor distancia Manhattan entre cualquier par– del conjunto formado por la posicion actual mas todas las esquinas pendientes.

$$h(n) = \max \left(\max_{c \in C_p} d_M(n, c), \max_{c_i, c_j \in C_p} d_M(c_i, c_j) \right)$$

Es mas informativa que la basica porque tambien tiene en cuenta que, para visitar dos esquinas pendientes lejanas entre si, Pac-Man tiene que recorrer al menos la distancia real entre ellas en algun momento del camino –no solo la distancia desde su posicion actual hasta la mas lejana de las dos–.

```

365 def cornersHeuristic(state, problem):
366     """
367     Actividad 8 -- Heurística propuesta (la que usa AStarCornersAgent).
368     Generaliza la básica:  $h(n)$  = diámetro Manhattan de {posición} union
369     {esquinas pendientes} (la mayor distancia entre cualquier par de ese
370     conjunto). Mas informativa que la básica; admisible y consistente por el
371     mismo argumento que foodHeuristic (ver docs/guia_codigos_clave.md).
372     Sin cache (a lo sumo 4 esquinas, recalculer es insignificante).
373     """
374     position, visited = state
375     corners = problem.corners
376     pendientes = [c for c, v in zip(corners, visited) if not v]
377     if not pendientes:
378         return 0
379
380     puntos = [position] + pendientes
381     maxDist = 0
382     for i in range(len(puntos)):
383         for j in range(i + 1, len(puntos)):
384             p1, p2 = puntos[i], puntos[j]
385             d = abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])
386             if d > maxDist:
387                 maxDist = d
388     return maxDist

```

Figura 8: cornersHeuristic (heurística propuesta, la que usa AStarCornersAgent) en pacman/searchAgents.py (líneas 365-389). Comparar con la Figura 7: aquí se recorren todos los pares de $\{\text{posición}\} \cup \text{pendientes}$, no solo posición→esquina.

A diferencia de foodHeuristic, aquí no se uso problem.heuristicInfo como cache: como mucho hay 4 esquinas (nunca mas de 6 pares), así que el costo de recalculer todas las distancias entre pares en cada llamada es insignificante –muy distinto al caso de FoodSearchProblem, donde puede haber decenas de alimentos y cientos de pares–. Cachear aquí añadiría la misma sobrecarga de diccionario que en la Actividad 11 no compenso, sin ningun beneficio real dado lo pequeno del conjunto.

8.4 Admisibilidad

Heurística básica. El costo real para visitar todas las esquinas pendientes es, como minimo, el costo de llegar hasta la esquina pendiente mas lejana –en algun momento del recorrido Pac-Man tiene que llegar hasta ella–, y la distancia Manhattan nunca sobreestima una distancia real en una cuadrícula con paredes (cada paso real cambia x o y en una unidad; la distancia Manhattan es el minimo numero de esos pasos sin contar paredes). Por lo tanto $h(n) \leq h^*(n)$.

Heurística propuesta. El recorrido que visita, en cualquier orden, la posición n y todas las esquinas de C_p tiene una longitud real que es, como minimo, la distancia real entre el par de puntos mas separado entre si de $\{n\} \cup C_p$ –en algun momento del recorrido Pac-Man pasa por ambos puntos de ese par–. Como la distancia Manhattan nunca sobreestima la distancia real, el diámetro en Manhattan de $\{n\} \cup C_p$ es una cota inferior valida: $h(n) \leq h^*(n)$.

Ambas cotas se verificaron tambien de forma empirica en el estado inicial de tinyCorners (ver el script de la Actividad 8 en experimentos/): $h_{\text{básica}}(\text{inicio}) = 6$ y $h_{\text{propuesta}}(\text{inicio}) = 12$, ambos por debajo del costo optimo real conocido (22, obtenido con UCS en la Actividad 7).

8.5 Consistencia

Una heurística consistente debe cumplir $h(n) \leq c(n, n') + h(n')$ para cualquier estado n y cualquiera de sus sucesores n' , además de $h(\text{meta}) = 0$.

Para cualquier sucesor n' de n (un paso ortogonal, costo 1), si el par de puntos que define $h(n)$ sigue intacto en n' (ninguno de los dos era la esquina que se acaba de visitar al pasar de n a n'), entonces por la desigualdad triangular de Manhattan $d_M(n, p) \leq d_M(n, n') + d_M(n', p) = 1 + d_M(n', p) \leq 1 + h(n')$ para el punto p correspondiente. Si uno de los dos puntos del par era justo la posición n (que cambia a n') o una esquina que se acaba de visitar, el argumento se reduce al caso trivial $d_M(n, n') = 1 \leq 1 + h(n')$, válido porque $h(n') \geq 0$ siempre. En consecuencia, $h(n) \leq c(n, n') + h(n')$ se cumple en todos los casos, y $h(\text{meta}) = 0$ (no quedan esquinas pendientes). Ambas heurísticas son, entonces, también consistentes.

8.6 Pregunta de análisis (consistencia)

Analice si la heurística propuesta para el problema de las esquinas puede considerarse consistente. Justifique su respuesta.

Sí, la heurística propuesta (y también la básica) es consistente. Como se muestra en la subsección anterior, para cualquier sucesor n' de n el valor de h solo puede cambiar en, como máximo, 1 unidad respecto a $h(n')$: el punto (esquina o posición) que definía el máximo en n o bien sigue presente en n' –y entonces la desigualdad triangular de Manhattan acota $d_M(n, p) \leq 1 + d_M(n', p) \leq 1 + h(n')$ –, o bien era la esquina que Pac-Man acaba de visitar al pasar de n a n' –y entonces el caso se reduce a $d_M(n, n') = 1 \leq 1 + h(n')$, válido porque $h(n') \geq 0$ siempre–. En ambos casos $h(n) \leq c(n, n') + h(n') = 1 + h(n')$, que es exactamente la definición de consistencia, y $h(\text{meta}) = 0$ porque al no quedar esquinas pendientes el diámetro del conjunto (un único punto, la posición final) es cero. Vale la pena notar que en A* con búsqueda en grafo (como la implementada en las Actividades 2-3) la consistencia es, de hecho, la propiedad más fuerte que realmente se necesita: consistencia implica admisibilidad, y además garantiza que el costo $g(n)$ con el que un estado se expande por primera vez ya es su costo óptimo, sin necesidad de reabrir nodos ya expandidos.

8.7 Procedimiento y resultados

Se ejecuto A* con las tres heurísticas sobre tinyCorners (ver el script de la Actividad 8 en experimentos/).

Cuadro 9: A* sobre CornersProblem con distintas heurísticas (tinyCorners)

Heurística	Costo	Longitud	Nodos expandidos	Tiempo (s)
$h(n) = 0$	22	22	295	0.001486
Básica	22	22	147	0.000832
Propuesta	22	22	119	0.000802

Las tres estrategias encuentran el mismo costo óptimo (22, el mismo que dio UCS en la Actividad 7), confirmando que ninguna de las dos heurísticas diseñadas sobreestima el costo real. La heurística básica reduce los nodos expandidos de 295 a 147 (49.8 % menos

que $h = 0$); la heurística propuesta reduce aun mas, a 119 (59.7 % menos que $h = 0$, y 19.0 % menos que la basica), confirmando que el termino adicional de distancia entre pares de esquinas pendientes si aporta informacion util a la busqueda.

8.8 Pregunta de análisis

Explique por qué su heurística para `CornersProblem` es admisible.

Porque en ambos casos el valor que devuelve la heurística es una distancia Manhattan (o el maximo de varias distancias Manhattan) entre puntos que, de todas formas, Pac-Man tiene que recorrer en algun momento para completar el objetivo –llegar a cada esquina pendiente, y en el caso de la heurística propuesta, tambien recorrer la distancia entre pares de esquinas pendientes entre si–. Como la distancia Manhattan es, por construccion, una cota inferior de cualquier distancia real en una cuadrícula con obstaculos (un camino real nunca es mas corto que la distancia en linea recta por cuadrícula, porque cada paso real cambia como maximo una coordenada en una unidad), el valor devuelto nunca puede ser mayor que el costo optimo real restante $h^*(n)$. No basta con decir “la heurística funciona” –lo que hace que sea admisible es, especificamente, que cada termino del maximo es una distancia Manhattan entre dos puntos que forman parte, necesariamente, de cualquier solucion real desde ese estado.

9 Actividad 9. Experimento comparativo (esquinas)

9.1 Procedimiento

Se ejecutaron las cuatro estrategias que pide la guia (UCS, $A^*+h=0$, A^* +heurística basica, A^* +heurística propuesta) sobre `tinyCorners`, el layout de referencia adoptado en la Actividad 7 (ver el hallazgo sobre `mediumCorners` en esa seccion). El script `experimentos/actividad9_corners_comparacion.py` corre las cuatro por separado (cada una con su propia instancia de `CornersProblem`, para que el contador `_expanded` de cada una no se mezcle con las demas) y verifica que las cuatro llegan al mismo costo optimo antes de reportar los resultados.

9.2 Resultados

Cuadro 10: Experimento comparativo sobre `CornersProblem` (`tinyCorners`)

Método	Costo	Expandidos	Tiempo (s)	Óptimo
UCS	22	295	0.002134	sí
$A^* + h=0$	22	295	0.001887	sí
A^* + heurística básica	22	147	0.001370	sí
A^* + heurística propuesta	22	119	0.001297	sí

Como es de esperarse, UCS y $A^*+h=0$ expanden exactamente el mismo numero de nodos (295): sin heurística, A^* se reduce a UCS. Las dos heurísticas diseñadas en la Actividad 8 reducen la exploracion sin perder optimalidad: las cuatro estrategias encuentran el mismo costo (22).

9.3 Factor de reducción de expansiones

$$R = \frac{N_{UCS}}{N_{A^*}} = \frac{295}{119} = 2,48$$

usando la heurística propuesta (la mas informada de las dos disenadas) como referencia de N_{A^*} . Esto significa que UCS expandio aproximadamente 2.48 veces mas estados que A^* +heurística propuesta para resolver el mismo problema con el mismo costo optimo. Si se usa la heurística basica en su lugar, $R = 295/147 = 2,01$: tambien reduce la exploracion a la mitad, pero menos que la propuesta.

9.4 Análisis

La relacion entre la calidad de la heurística y el numero de nodos expandidos es directa: entre mas informacion aporte la heurística sobre el costo real restante (sin dejar de ser una cota inferior, es decir, sin perder admisibilidad), menos nodos necesita expandir A^* para encontrar la misma solucion optima. Esto se observa con claridad en la progresion $h = 0 \rightarrow$ basica $\rightarrow$ propuesta: cada heurística anadida es al menos tan informada como la anterior (la propuesta siempre devuelve un valor mayor o igual que la basica, porque incluye el mismo termino de distancia a la esquina mas lejana mas el termino adicional de distancia entre pares de esquinas pendientes), y en cada paso el numero de nodos expandidos baja: $295 \rightarrow 147 \rightarrow 119$. Ninguna de las dos heurísticas hace que A^* pierda optimalidad (las cuatro estrategias llegan al mismo costo 22) —exactamente lo que garantiza que una heurística sea admisible—, pero si reducen sustancialmente cuanto tiene que explorar el algoritmo para encontrarla.

10 Actividad 10. Búsqueda de todos los alimentos

10.1 El problema

`FoodSearchProblem` ya viene completamente implementado en `searchAgents.py` (`getStartState`, `isGoalState`, `getSuccessors` y `getCostOfActions`): no hubo que escribir código nuevo en esta actividad. El objetivo es entender el problema y establecer una línea base antes de diseñar heurísticas en la Actividad 11.

El estado ya no es solo la posición de Pac-Man: es el par $s = (p, F)$, donde p es la posición (x, y) y F es una `Grid` (matriz booleana) con el estado —presente o consumido— de cada alimento del laberinto:

```

396 class FoodSearchProblem:
397     """
398     A search problem associated with finding the a path that collects all of the
399     food (dots) in a Pacman game.
400
401     A search state in this problem is a tuple ( pacmanPosition, foodGrid ) where
402     pacmanPosition: a tuple (x,y) of integers specifying Pacman's position
403     foodGrid:      a Grid (see game.py) of either True or False, specifying remaining food
404     """
405     def __init__(self, startingGameState):
406         self.start = (startingGameState.getPacmanPosition(), startingGameState.getFood())
407         self.walls = startingGameState.getWalls()
408         self.startingGameState = startingGameState
409         self._expanded = 0
410         self.heuristicInfo = {} # A dictionary for the heuristic to store information
411
412     def getStartState(self):
413         return self.start
414
415     def isGoalState(self, state):
416         return state[1].count() == 0
417

```

Figura 9: Estado y isGoalState de FoodSearchProblem en pacman/searchAgents.py (líneas 396-417).

La meta se alcanza cuando ya no queda ningún alimento por consumir (`foodGrid.count() == 0`), sin importar en qué posición esté Pac-Man.

10.2 Por qué crece tanto el espacio de estados

Cada alimento puede estar en dos condiciones: presente o consumido. Si el laberinto tiene F alimentos, el número de configuraciones posibles del Grid de comida es, conceptualmente, 2^F –y el estado completo es el producto de eso por el número de posiciones posibles de Pac-Man–. Esto es exactamente lo que dice la guía, y lo confirmamos experimentalmente: intentamos correr `uniformCostSearch` sobre `smallClassic` (55 alimentos) con un límite de 45 segundos y **no terminó**. 2^{55} es un número astronómicamente más grande que cualquier espacio de estados manejado en las Actividades 2-9 (donde el estado era solo una posición, o una posición más 4 bits de esquinas visitadas – $2^4 = 16$ combinaciones como máximo). Por esta razón, esta actividad y la 11 se trabajan sobre `tinySearch` (1 alimento) y `testClassic` (8 alimentos), donde UCS puro sigue siendo viable como línea base.

10.3 Procedimiento y resultados

Se ejecutaron UCS y A* con `nullHeuristic` ($h(n) = 0$) sobre `tinySearch` y `testClassic` (ver `experimentos/actividad10_food_baseline.py`), replicando la misma verificación de la Actividad 4 (con $h(n) = 0$, A* debe coincidir exactamente con UCS) pero ahora sobre `FoodSearchProblem`.

Cuadro 11: Línea base de UCS y A*+ $h(n)=0$ sobre FoodSearchProblem

Layout (alimentos)	Método	Costo	Expandidos	Tiempo (s)
tinySearch (1)	UCS	8	16	0.000393
tinySearch (1)	A*+ $h(n)=0$	8	16	0.000303
testClassic (8)	UCS	16	2598	0.063502
testClassic (8)	A*+ $h(n)=0$	16	2598	0.061340

En ambos layouts, UCS y $A^*+h(n)=0$ encuentran el mismo costo óptimo y expanden exactamente el mismo número de nodos –la misma consecuencia algebraica de $f(n) = g(n) + 0 = g(n)$ ya explicada en la Actividad 4–. Nótese el salto de 16 a 2598 nodos expandidos al pasar de 1 a solo 8 alimentos: es la primera evidencia numérica, dentro de un caso todavía manejable, de la explosión combinatoria que describe la guía, y es exactamente la motivación para diseñar una heurística no trivial en la Actividad 11 –sin ella, layouts con más alimentos que `testClassic` dejan de ser viables, como mostró el intento fallido sobre `smallClassic`.

11 Actividad 11. Diseño de heurísticas para la búsqueda de alimentos

11.1 Heurística 1: distancia al alimento más lejano

Es exactamente la fórmula que sugiere la guía como primera aproximación: $h(n) = \max_{f \in F} d_M(n, f)$, la distancia Manhattan desde la posición actual hasta el alimento restante más lejano.

```

497 def foodHeuristicV1(state, problem):
498     position, foodGrid = state
499     foodList = foodGrid.asList()
500     if not foodList:
501         return 0
502     return max(abs(position[0]-f[0]) + abs(position[1]-f[1]) for f in foodList)
503
504
505 def foodHeuristic(state, problem):
506     position, foodGrid = state
507     foodList = foodGrid.asList()
508     if not foodList:
509         return 0
510
511     if 'foodPairDistances' not in problem.heuristicInfo:
512         allFood = problem.start[1].asList()
513         distancias = {}
514         for i in range(len(allFood)):
515             for j in range(i + 1, len(allFood)):
516                 f1, f2 = allFood[i], allFood[j]
517                 d = abs(f1[0]-f2[0]) + abs(f1[1]-f2[1])
518                 distancias[(f1, f2)] = d
519                 distancias[(f2, f1)] = d
520             problem.heuristicInfo['foodPairDistances'] = distancias
521
522     distanciasComida = problem.heuristicInfo['foodPairDistances']
523     maxDist = 0
524     for f in foodList:
525         d = abs(position[0]-f[0]) + abs(position[1]-f[1])

```

Figura 10: `foodHeuristicV1` en `pacman/searchAgents.py` (líneas 452-464).

Admisibilidad: el costo real para recoger toda la comida restante es, como mínimo, el costo de llegar hasta el alimento más lejano –en algún momento del recorrido Pac-Man tiene que llegar hasta él–, y la distancia Manhattan nunca sobreestima una distancia real en una cuadrícula con paredes. Por lo tanto $h(n) \leq h^*(n)$ siempre.

Consistencia: para cualquier sucesor n' de n (un paso ortogonal, costo 1), si el alimento más lejano de n sigue presente en n' (no fue el que se acaba de comer), la desigualdad triangular de Manhattan da $d_M(n, f^*) \leq d_M(n, n') + d_M(n', f^*) = 1 + d_M(n', f^*) \leq$

$1 + h(n')$. Si el alimento más lejano de n era justo el que estaba en n' (se acaba de comer), $d_M(n, n') = 1 \leq 1 + h(n')$ porque $h(n') \geq 0$ siempre. En ambos casos $h(n) \leq 1 + h(n')$.

11.2 Heurística 2: diámetro Manhattan de posición + comida restante

Generaliza la Heurística 1: en vez de mirar solo la distancia de Pac-Man al alimento más lejano, considera el **diámetro** (la mayor distancia Manhattan entre cualquier par) del conjunto formado por la posición actual *más* todos los alimentos restantes:

$$h(n) = \max \left(\max_{f \in F} d_M(n, f), \max_{f_i, f_j \in F} d_M(f_i, f_j) \right)$$

Es más informativa que la Heurística 1 porque también considera que, para visitar dos alimentos lejanos entre sí, Pac-Man tiene que recorrer al menos la distancia real entre ellos en algún momento del camino —no solo la distancia desde su posición actual hasta el más lejano de los dos—.

```

497 def foodHeuristicV1(state, problem):
498     position, foodGrid = state
499     foodList = foodGrid.asList()
500     if not foodList:
501         return 0
502     return max(abs(position[0]-f[0]) + abs(position[1]-f[1]) for f in foodList)
503
504
505 def foodHeuristic(state, problem):
506     position, foodGrid = state
507     foodList = foodGrid.asList()
508     if not foodList:
509         return 0
510
511     if 'foodPairDistances' not in problem.heuristicInfo:
512         allFood = problem.start[1].asList()
513         distancias = {}
514         for i in range(len(allFood)):
515             for j in range(i + 1, len(allFood)):
516                 f1, f2 = allFood[i], allFood[j]
517                 d = abs(f1[0]-f2[0]) + abs(f1[1]-f2[1])
518                 distancias[(f1, f2)] = d
519                 distancias[(f2, f1)] = d
520         problem.heuristicInfo['foodPairDistances'] = distancias
521
522     distanciasComida = problem.heuristicInfo['foodPairDistances']
523     maxDist = 0
524     for f in foodList:
525         d = abs(position[0]-f[0]) + abs(position[1]-f[1])

```

Figura 11: foodHeuristic (Heurística 2, con caché en problem.heuristicInfo) en pacman/searchAgents.py (líneas 465-488, parte 1 de 2). El bloque if 'foodPairDistances' not in problem.heuristicInfo es el caché discutido más abajo.

```

522     distanciasComida = problem.heuristicInfo['foodPairDistances']
523     maxDist = 0
524     for f in foodList:
525         d = abs(position[0]-f[0]) + abs(position[1]-f[1])
526         if d > maxDist: maxDist = d
527     for i in range(len(foodList)):
528         for j in range(i+1, len(foodList)):
529             d = distanciasComida[(foodList[i], foodList[j])]
530             if d > maxDist: maxDist = d
531     return maxDist

```

Figura 12: foodHeuristic (líneas 490-502, parte 2 de 2): uso del caché ya calculado (distanciasComida) para obtener maxDist y el return final.

Admisibilidad: el recorrido que visita, en cualquier orden, la posición n y todos los alimentos de F tiene una longitud real que es, como mínimo, la distancia real entre el par de puntos (de $\{n\} \cup F$) más separado entre sí –en algún momento del recorrido Pac-Man pasa por ambos puntos de ese par–. Como la distancia Manhattan nunca sobreestima la distancia real, el diámetro en Manhattan de $\{n\} \cup F$ es una cota inferior válida: $h(n) \leq h^*(n)$.

Consistencia: el mismo argumento de desigualdad triangular de la Heurística 1 se aplica aquí a cualquier par de puntos de $\{n, n'\} \cup F$. El caso más delicado es cuando se acaba de comer el alimento en n' (es decir, $F' = F \setminus \{n'\}$ tras el paso): como $\{n\} \cup F = \{n, n'\} \cup F'$, para cualquier par que incluya a n y a algún $q \in \{n'\} \cup F'$ se cumple $d_M(n, q) \leq d_M(n, n') + d_M(n', q) = 1 + d_M(n', q) \leq 1 + h(n')$ (porque $q \in \{n'\} \cup F'$ acota por el máximo); si el par no incluye a n , ya está dentro de $\{n'\} \cup F'$ y su distancia es, por definición, $\leq h(n')$. Por lo tanto $h(n) \leq 1 + h(n')$ en todos los casos, y $h(\text{meta}) = 0$ porque con $F = \emptyset$ el conjunto $\{n\}$ es un único punto (diámetro 0).

11.3 Optimización con problem.heuristicInfo: qué se cachea y por qué

El cálculo que se repite en *cada* llamada a la heurística (una por cada nodo expandido, potencialmente miles de veces) es la distancia Manhattan entre cada *par* de alimentos. Pero las posiciones de los alimentos nunca cambian durante la búsqueda –solo si siguen presentes o no–, así que esas distancias se calculan una sola vez (para todos los pares del layout inicial, usando problem.start[1], el foodGrid original) y se guardan en problem.heuristicInfo['foodPairDistances']; las llamadas siguientes reutilizan ese diccionario en vez de recalculan la fórmula de distancia para el mismo par de alimentos.

11.4 Procedimiento y resultados

Se ejecutó A* con las tres heurísticas ($h(n) = 0$, Heurística 1, Heurística 2) sobre tinySearch y testClassic (ver experimentos/actividad11_food_heuristic.py).

Cuadro 12: Comparación de heurísticas de comida sobre testClassic (8 alimentos)

Heurística	Costo	Longitud	Expandidos	Tiempo (s)
$h(n) = 0$	16	16	2598	0.070222
Heurística 1	16	16	702	0.024714
Heurística 2	16	16	383	0.014872

En `tinySearch` (1 alimento) las tres coinciden en costo (8) y las dos heurísticas empatan en 8 nodos expandidos frente a los 16 de $h(n) = 0$ –con un solo alimento no hay ninguna pareja de alimentos que comparar, así que la Heurística 2 se reduce exactamente a la Heurística 1–. La diferencia real aparece en `testClassic`: la Heurística 1 ya reduce drásticamente la exploración frente a $h(n) = 0$ ($2598 \rightarrow 702$, factor $R \approx 3,70$), y la Heurística 2 la reduce aún más ($2598 \rightarrow 383$, factor $R \approx 6,78$; 319 nodos menos que la Heurística 1), confirmando que considerar también la distancia entre pares de alimentos –no solo la distancia desde Pac-Man– aporta información adicional real, no solo teórica.

11.5 El reto del caché: qué encontramos (y por qué es un resultado honesto, no un error)

Comparamos el tiempo de A*+Heurística 2 con caché (`foodHeuristic`) contra una versión idéntica pero sin caché (`foodHeuristicV2SinCache`, que recalcula las distancias entre pares de alimentos restantes en cada llamada), promediando 5 corridas por versión en dos layouts:

Cuadro 13: Heurística 2 con caché vs. sin caché (tiempo promedio de 5 corridas)

Layout (alimentos)	Expandidos	Tiempo con caché (s)	Tiempo sin caché (s)
testClassic (8)	383	0.015458	0.014835
capsuleClassic (23)	2185	0.207050	0.193121

El resultado es contraintuitivo pero reproducible: la versión *sin* caché fue, en ambos casos, ligeramente *más rápida* (entre 4% y 7%), no más lenta. La explicación no es que el caché esté mal implementado –los nodos expandidos son idénticos en ambas versiones, porque es exactamente la misma heurística–, sino que el cálculo que se está cacheando es demasiado barato para que valga la pena: son sumas y restas de enteros ($\text{abs}(f1[0]-f2[0]) + \text{abs}(f1[1]-f2[1])$), mientras que consultar el diccionario `problem.heuristicInfo['foodPairDistances']` implica construir una tupla (`f1`, `f2`) como llave y calcular su hash en cada consulta. En este problema, con a lo sumo un par de decenas de alimentos, el costo de esa búsqueda en el diccionario termina siendo comparable –o incluso mayor– al costo de simplemente recalcular la resta. El caché con `problem.heuristicInfo` sigue siendo la herramienta correcta que sugiere la guía, pero su beneficio depende de que el cálculo evitado sea realmente costoso (por ejemplo, una distancia real de laberinto calculada con BFS en vez de una resta Manhattan); aquí, honestamente, no lo es. Dejamos ambas versiones en el código (`foodHeuristic` con caché se usa en `AStarFoodSearchAgent`, y `foodHeuristicV2SinCache` existe solo para esta comparación) precisamente para poder mostrar esta medición en vivo si se pide en la sustentación.

12 Reto: Competencia de heurísticas

La guía pide que cada grupo diseñe una heurística propia para uno de los dos problemas (`CornersProblem` o `FoodSearchProblem`) y compita contra las heurísticas de los demás grupos, minimizando el número de nodos expandidos sin perder la optimalidad de la solución.

Estado actual: pendiente de decisión del grupo. Ya se cuenta con material completo para competir con cualquiera de los dos problemas, con toda la documentación que exige la guía (nombre, idea, fórmula, implementación, admisibilidad, consistencia, costo, nodos expandidos, tiempo) ya redactada en las secciones correspondientes:

- **CornersProblem:** `cornersHeuristic` (Actividad 8), diámetro Manhattan de $\{\text{posición}\} \cup \{\text{esquinas pendientes}\}$: 119 nodos expandidos sobre `tinyCorners`, frente a 295 con $h(n) = 0$ (factor $R = 2,48$, Actividad 9).
- **FoodSearchProblem:** `foodHeuristic` (Actividad 11, Heurística 2) –el mismo patrón de diámetro Manhattan, aplicado a $\{\text{posición}\} \cup \{\text{comida restante}\}$ –, 383 nodos expandidos sobre `testClassic` frente a 2598 de $h(n) = 0$ (factor $R = 6,78$).

Falta únicamente que el grupo decida cuál de las dos presentar a la competencia entre grupos; una vez tomada esa decisión, esta sección se completa citando directamente la subsección correspondiente (Actividad 8 o Actividad 11) como la documentación formal del reto, sin necesidad de rehacer ningún cálculo.

13 Resultados generales

La siguiente tabla consolida los principales resultados experimentales de todas las actividades. Cada método se evaluó sobre el layout de referencia adoptado para ese tipo de problema a lo largo del laboratorio (ver la nota al pie de la tabla): `mediumClassic` para los métodos de posición simple (Actividades 2, 4, 5 y 6), `tinyCorners` para el problema de las esquinas (Actividades 7-9) y `testClassic` para el problema de recolección de alimentos (Actividades 10-11).

Cuadro 14: Resultados generales consolidados

Experimento	Costo	Expandidos	Tiempo (s)
UCS	12	69	0.000251
A* + heurística nula	12	69	0.000262
A* + Manhattan	12	15	0.000066
A* + Euclidiana	12	16	0.000070
A* + Corners Heuristic	22	119	0.001297
A* + Food Heuristic	16	383	0.014872

Nota: las filas de UCS, heurística nula, Manhattan y Euclidiana corresponden a `mediumClassic`; la fila de Corners Heuristic corresponde a `tinyCorners` (heurística propuesta de la Actividad 8, la que usa `AStarCornersAgent`); la fila de Food Heuristic corresponde a `testClassic` (Heurística 2 de la Actividad 11, la que usa `AStarFoodSearchAgent`). Los costos no son comparables entre filas de distinto layout –cada uno tiene su propio costo óptimo–.

Lo que sí es comparable dentro de cada bloque es la reducción de nodos expandidos frente a su propia línea base de UCS o heurística nula en el mismo layout (ver las secciones de cada actividad para el detalle completo, incluidas las heurísticas intermedias que se omiten aquí por brevedad –básica en esquinas, Heurística 1 en comida–).

14 Análisis final

A continuación se responden las diez preguntas de análisis final que pide la guía, con base en los resultados experimentales obtenidos a lo largo de todas las actividades.

1. ¿Cuál es la diferencia fundamental entre UCS y A*?

UCS selecciona para expansión el estado con menor $f(n) = g(n)$, es decir, únicamente el costo acumulado real desde el inicio. A* selecciona el estado con menor $f(n) = g(n) + h(n)$, incorporando además una estimación heurística del costo restante hasta la meta. Esa información adicional es lo que permite a A* orientar la búsqueda hacia la meta y expandir menos estados que UCS, sin perder la garantía de optimalidad siempre que $h(n)$ sea admisible.

2. ¿Qué función cumple $g(n)$?

$g(n)$ es el costo acumulado real (conocido, no estimado) del camino recorrido desde el estado inicial hasta el estado n . Es la misma cantidad que UCS usa como única prioridad.

3. ¿Qué función cumple $h(n)$?

$h(n)$ es una estimación del costo que falta recorrer desde n hasta un estado objetivo. No es un valor conocido con exactitud (salvo en el caso trivial $h(n) = 0$): es una heurística, calculada con información parcial del problema (por ejemplo, distancia Manhattan, distancia Euclidiana, o el diámetro de posiciones pendientes por visitar en los problemas de esquinas y comida).

4. ¿Qué ocurre cuando $h(n) = 0$?

A* se reduce exactamente a UCS, porque $f(n) = g(n) + 0 = g(n)$: la prioridad usada para elegir el siguiente estado a expandir es idéntica en ambos algoritmos. Esto se confirmó empíricamente en dos problemas distintos: en `mediumClassic`, UCS y A*+ $h(n)=0$ expanden exactamente 69 nodos cada uno (Actividad 4); en `tinyCorners`, UCS y A*+ $h(n)=0$ expanden exactamente 295 nodos cada uno (Actividad 9). En ambos casos el costo óptimo encontrado también es idéntico.

5. ¿Cuál presentó mejores resultados: Manhattan o Euclidiana?

Manhattan, con menos nodos expandidos (15 contra 16 en `mediumClassic`, Actividad 6), aunque ambas heurísticas encuentran el mismo costo óptimo (12). La razón es que Pac-Man solo se mueve en las cuatro direcciones cardinales (nunca en diagonal), y matemáticamente $h_E(n) \leq h_M(n)$ siempre (la distancia en línea recta nunca es mayor que la distancia recorrida por una cuadrícula). Por lo tanto Manhattan es una cota más ajustada –más informada– sin dejar de ser admisible, y una heurística admisible más informada expande menos nodos (ver pregunta 6).

6. ¿Una heurística más grande siempre es mejor? Justifique.

Solo si sigue siendo admisible, es decir, si sigue cumpliendo $h(n) \leq h^*(n)$. Entre dos heurísticas admisibles, la que devuelve valores mayores (más informada) expande menos o igual número de nodos que la menos informada, sin perder optimalidad –esto se demostró repetidamente en este laboratorio: en el problema de esquinas, la heurística propuesta (diámetro) siempre devuelve un valor mayor o igual que la

básica, y expande menos nodos (119 contra 147, Actividad 8); en el problema de comida, la Heurística 2 (diámetro) expande menos que la Heurística 1 (383 contra 702 en `testClassic`, Actividad 11). Si la heurística dejara de ser admisible (sobreestimara en algún estado), “más grande” dejaría de ser sinónimo de “mejor” –véase la pregunta 7–.

7. ¿Por qué una heurística que sobreestima puede ser problemática?

Porque rompe la condición $h(n) \leq h^*(n)$ que garantiza la optimalidad de A^* . Si en algún estado $h(n)$ es mayor que el costo real restante, A^* puede descartar –o posponer indefinidamente– un camino que en realidad era parte de la solución óptima, porque su $f(n) = g(n) + h(n)$ aparenta ser peor de lo que realmente es. El algoritmo puede terminar devolviendo una solución de costo mayor al óptimo sin que nada en su ejecución indique que ocurrió un error: el resultado se ve igual de “válido” que uno correcto, pero no lo es.

8. ¿Por qué el estado de `CornersProblem` necesita almacenar más información que únicamente (x,y) ?

Porque la prueba de objetivo de este problema no depende solo de dónde está Pac-Man, sino de qué ha hecho para llegar ahí –específicamente, cuáles de las 4 esquinas ya visitó–. Si el estado fuera solo la posición, dos situaciones con la misma posición pero distinto progreso (unas esquinas visitadas y otras no) se confundirían en un único estado, y un algoritmo de búsqueda podría descartar como “ya explorado” un estado que en realidad todavía necesitaba explorarse con otra combinación de esquinas pendientes, perdiendo la garantía de encontrar la solución óptima –o incluso de encontrar alguna solución– (desarrollado en detalle en la Actividad 7).

9. ¿Por qué `FoodSearchProblem` presenta un espacio de estados considerablemente mayor?

Porque el estado incluye qué alimentos siguen presentes, y cada uno de los F alimentos puede encontrarse en dos condiciones ($\{\text{presente, consumido}\}$), por lo que el número de configuraciones posibles de comida crece como 2^F , multiplicado además por cada posición posible de Pac-Man. Esto se confirmó empíricamente: con apenas 8 alimentos (`testClassic`), $UCS/A^*+h=0$ ya expanden 2598 nodos (Actividad 10); con 55 alimentos (`smallClassic`) la búsqueda no terminó en 45-60 segundos, ilustrando de forma directa la explosión combinatoria de 2^F .

10. ¿Qué relación encontró entre la calidad de la heurística y el número de nodos expandidos?

Una relación inversa y consistente en los tres problemas trabajados: entre más informada (más cercana al costo real, sin dejar de ser una cota inferior) es la heurística, menos nodos necesita expandir A^* para encontrar la misma solución óptima. Se observó la misma progresión en dos problemas distintos: en esquinas, $h = 0 \rightarrow$ básica $\rightarrow$ propuesta redujo la exploración de 295 a 147 a 119 nodos (Actividad 9, factor de reducción $R = 2,48$ con la propuesta); en comida, $h = 0 \rightarrow$ Heurística 1 $\rightarrow$ Heurística 2 redujo de 2598 a 702 a 383 nodos sobre `testClassic` (Actividad 11). En ambos casos el costo óptimo permaneció idéntico en todas las estrategias, confirmando que la reducción de exploración no tiene ningún costo en la calidad de la solución –siempre que la heurística conserve la admisibilidad–.

15 Conclusiones

El objetivo de este laboratorio no era simplemente lograr que Pac-Man llegara al objetivo, sino comprender cómo una función heurística introduce conocimiento del problema dentro del proceso de búsqueda, y cómo ese conocimiento se traduce en una reducción medible del espacio explorado sin sacrificar la calidad de la solución.

Los resultados experimentales de las once actividades sostienen esa idea de forma consistente en los tres problemas trabajados (búsqueda simple, esquinas y comida): en todos los casos, A^* con $h(n) = 0$ se comportó de forma idéntica a UCS (mismo costo, mismo número de nodos expandidos), confirmando que ambos algoritmos comparten el mismo esqueleto y que la única diferencia real está en la información que aporta $h(n)$. A partir de ahí, cada heurística admisible que se fue incorporando –Manhattan y Euclidiana en búsqueda simple; la heurística básica y la propuesta en el problema de esquinas; las Heurísticas 1 y 2 en el problema de comida– redujo el número de nodos expandidos sin cambiar el costo óptimo encontrado. La progresión fue especialmente clara en los dos problemas donde se diseñaron heurísticas propias: en esquinas, de 295 a 147 a 119 nodos (tinyCorners, factor de reducción $R = 2,48$ con la heurística propuesta); en comida, de 2598 a 702 a 383 nodos (testClassic). En ambos casos la heurística más informada –la que considera el diámetro entre la posición actual y los puntos pendientes por visitar, en vez de solo la distancia al más lejano– fue también la que menos nodos expandió, ilustrando de forma directa la relación central de la búsqueda informada:

mejor información $\implies$ menor exploración del espacio de estados,

siempre que la función heurística preserve las propiedades de admisibilidad (y, cuando se pudo demostrar, consistencia) necesarias para garantizar la optimalidad de A^* .

El laboratorio también dejó hallazgos que no estaban en el guión pero que enriquecieron el trabajo: un bug real de desempate (*tie-breaking*) en la cola de prioridad que afectaba tanto a nuestra implementación como a la de un compañero de grupo, corregido y verificado en ambas; dos layouts (bigMaze y mediumCorners) descartados tras confirmar, con el propio código del profesor, que tenían defectos de conectividad ajenos a nuestra implementación; y un hallazgo honesto sobre el caché con `problem.heuristicInfo`, que en nuestro caso no mejoró el tiempo de ejecución –la aritmética evitada resultó más barata que la consulta al diccionario–, mostrando que optimizar no siempre significa cachear, y que vale la pena medir antes de asumir que una técnica estándar va a ayudar en un contexto concreto.

En conjunto, el trabajo confirma experimentalmente que diseñar una buena heurística es, en esencia, decidir cuánto se puede saber de antemano sobre la estructura del problema –la geometría de la cuadrícula, las metas pendientes, la distancia entre ellas– sin llegar a sobreestimar nunca el costo real restante.

Referencias

- [1] Universidad Sergio Arboleda (2026). *Guia de Laboratorio: Busqueda Informada con Pac-Man*. Asignatura Inteligencia Artificial (SIST5036).
- [2] Stanford University (2023). *CS221: Artificial Intelligence – Pac-Man Assignment*. <https://stanford-cs221.github.io/spring2023/assignments/pacman/index.html>
- [3] Russell, S., & Norvig, P. (2022). *Artificial Intelligence: A Modern Approach* (4th ed.).